

RESEARCH DEBRIEF • THAMMASATRESEARCH

Audit & Rerun: Adversarial Detection on Brain Tumor MRI

A summary for our discussion. The short version: your research is **better than the original numbers made it look** — but one leak in the process contaminated the final results table. We traced the root cause, reran the entire chain under a verifiable pipeline, and found a new result strong enough to become the centerpiece of the paper.

2026-09-02 dataset: Brain Tumor MRI (5,600 / 1,600)

rerun: RTX 3070 • TF 2.20.0 • seed 42 every number backed by CSV + log + SHA-256

TL;DR

Three sentences to know

THE PROBLEM

The classifier file was retrained over

Experiment 2's final test mixed the old model's detector with the new model's features — the original test table is invalid (a process bug, not a flaw in the method).

THE RERUN

A fully verifiable new pipeline

All 7 stages in ~20 GPU minutes — every seed fixed, canonical models never overwritten, SHA-256 checked before every stage. The old failure mode can no longer happen.

THE NEW FINDING

Thresholds don't transfer across collections

But just 400 clean images from the deployment domain pin the FPR on target — a contribution more interesting than the detector's raw numbers.

01 • WHAT HAPPENED

How 1,331 became 1,250

Reconstructed from file timestamps, the metadata inside the `.keras` file, and a confirming evaluation against the actual model:

- Aug 23
First classifier training — clean test accuracy 83.19% (1,331/1,600 correct), followed by the FGSM sweep
Every result after this point depends on this model.
- Aug 27
Experiment 1 — detector trained on MobileNetV2 features + final test (all numbers consistent with 1,331)
- Aug 29–30
Experiment 2 — consistency detector trained + threshold selected on validation (FPR 9.55%) — still on the old classifier
- Sep 1, 10:08 UTC
`baseline_mobilenetv2.keras` was retrained and overwritten
The file's own metadata says `date_saved: 2026-09-01@10:08:51`. The new model reached only 78.13% (1,250/1,600) — the training notebook never fixed TensorFlow's seed, so retraining produced a different model (meningioma recall collapsed 73% → 49%).
- Sep 1, 10:58 UTC
Experiment 2 was re-run — its resume logic skipped training and threshold selection (saved files existed) but re-executed the final test step
Result: a detector and threshold fitted to the old model, evaluated on the new model's features → a mixed-era test table (the 15.2% FPR seen then was partly this).
- Sep 1 (verification)
Confirmed by direct evaluation — loading the current model file and evaluating the Testing set yields **exactly 1,250**, matching the v2 table · the original 83.19% model no longer exists on disk

The key point: this is a process bug, not a research flaw

Experiment 2's methodology (successful-attack-only labels + consistency features + validation-selected threshold) is well designed — remeasured with all artifacts from the same era, its results are **substantially better** than the contaminated table suggested.

02 • THE RERUN

A pipeline where every number traces back to its source

The notebooks were rewritten as a 7-stage script pipeline (`ThammasatResearch/rerun/` in the repo, executed on a GPU machine) with every leak closed, then everything was remeasured from classifier training through the final test table:

Key metric	Old era (Aug)	Mixed-era table	Clean rerun
Clean test accuracy	83.19%	78.13%	81.88%
Correct predictions (of 1,600)	1,331	1,250	1,310
Attack success at $\epsilon=0.5$	90.6%	84.1%	88.8%
Attack success at $\epsilon=1.0$	96.0%	93.7%	95.0%

"Mixed-era table" is the original v2 test file, now retired. The rerun sits slightly below the old era because op determinism is enabled — the price of getting identical numbers on every re-run.

03 • MAIN FINDING

Thresholds calibrated on Training data fail on Testing – and the fix

The design target was $FPR \leq 10\%$. We tried three calibration strategies. The results show the problem is not leakage: the dataset's Training/ and Testing/ folders are **different collections** – clean test images score systematically higher in the detector's upper tail ($p90 = 0.355$ vs 0.151):

Threshold selection method	Claimed FPR	Actual test FPR	E
Full 1,120-image validation set (original v2)	$\leq 10\%$	12.9% missed	
Calibration split disjoint from model selection (v2b)	$\leq 10\%$	13.6% missed	
400 clean images from the Testing domain	$\leq 10\%$	9.8% on target	

The third method uses clean images only – no attack examples needed. That is deployment-realistic for a hospital setting, and leaves a 1,200-image eval set.

Why this is good news

"The detector generalizes well to unseen ϵ , but its operating point does not transfer across collections – and a small calibration set from the deployment domain fixes it" is a methodological contribution that anyone deploying an adversarial detector on medical imaging can use. It is more interesting than reporting a clean 10% would have been.

04 • HONEST NUMBERS

Detection rates at a true 10% FPR (1,200 eval images)

ϵ (0–255 scale)	Status	Detection	ROC-AUC	PR-AUC
0.01	known	54.2%	0.887	0.080
0.05	unseen	70.3%	0.923	0.535
0.10	known	87.6%	0.962	0.858
0.25	unseen	97.9%	0.988	0.979
0.50	known	99.0%	0.995	0.993
1.00	unseen	98.7%	0.995	0.994

"Unseen" rows are ϵ values the detector never saw in training — they sit on the same trend as the known ones, i.e. it generalizes from perturbation magnitude, not memorization.

The low- ϵ numbers are lower than before (e.g. 70.3% instead of 81.6% at $\epsilon=0.05$) — **the detector did not get worse**. This is the true price once false alarms are actually held at the claimed 10%; the old figures were "bought" with ~36% more false alarms than claimed. At $\epsilon=0.01$, PR-AUC is just 0.08 (only 24 positives against ~1,200 cleans) — the paper must state plainly that tiny attacks remain essentially undetectable, and always quote PR-AUC alongside ROC-AUC at small ϵ .

05 • LESSONS

Four workflow changes (all already built into the new scripts)

1 Seed everything

The original notebook only had `random.seed(42)` — it needs `tf.keras.utils.set_random_seed` + `enable_op_determinism()` so retraining reproduces the same model.

2 Never overwrite a canonical model

Save versioned files (`classifier_seed42.keras`); the scripts refuse to overwrite an existing model. This is the single rule that would have prevented the Sep 1 incident.

3 Fail-closed guards at every stage

Each stage verifies the model's SHA-256 and clean accuracy against `manifest.json` before doing any work — mixed-era runs now fail immediately instead of passing silently (Experiment 2 had no such guard, which is how the contamination slipped through).

4 Every number must trace to an artifact

The manifest records hashes, accuracies, and TF versions for every run; full logs are kept per stage — the same anti-fabrication standard the lab applies everywhere.

06 • NEXT

Proposed next steps

1 — Add a PGD attack · The whole study currently rests on FGSM alone, which will be a reviewer's first question. Testing the FGSM-trained detector against PGD attacks it has never seen would be the strongest generalization test in the paper — the script skeleton is ready and runs in about half an hour.

2 — Write the paper under the new framing · Lead with AUCs (threshold-free) as the primary claim · quote detection rates at the measured FPR · make the three-row FPR-transfer table the headline contribution · state the caveats plainly: the eval set is 75% of the original test set, and calibration uses clean images only (the latter is a strength, not a weakness).

3 — Recover the old model (if still possible) · Try Google Drive → right-click `baseline_mobilenetv2.keras` → Manage versions. Recovering it would make the August-era results reproducible again, but it is not blocking — the new pipeline fully replaces them.

Artifacts: GPU machine `~/ThammasatResearch/{models, results, results/_logs}` · scripts: `repo`
`ThammasatResearch/rerun/` · dataset checksum `8828...fdb4` · new classifier `3fab...b1de` · detector v2b `606a...a2b2` · deployment threshold 0.3612